ĐỀ THI – TH TRÍ TUỆ NHÂN TẠO

Trường Đại học Công Thương TP.HCM – Khoa CNTT
Bộ môn: Khoa học Dữ liệu

Thông tin	
Họ và tên	*(Nguyễn Thanh Tân)*
MSSV	*(2045240269)*
Lớp	*(TTNT02)*
Thời gian	90 phút
Tổng điểm	10 điểm (Câu 1: 2đ | Câu 2: 4đ | Câu 3: 4đ)

> **Lưu ý:** Sinh viên được sử dụng tài liệu. Không được trao đổi trong giờ thi. Nộp bài dưới dạng file `.pdf`.

```
# Cell nay tai anh de thi - chay TRUOC KHI lam bai
from IPython.display import Image, display
import os

IMGS = ['cau1_code_lan3.png', 'cau2_graph_lan3.png', 'cau3_code_lan3.png']
ok = all(os.path.exists(f) for f in IMGS)
print('OK - Anh de thi san sang.' if ok else 'CHUA CO ANH - upload 3 file PNG vao thu muc nay.')
```

```
OK - Anh de thi san sang.
```

CÂU 1 (2 ĐIỂM) – Quay Lui: Sinh Hoán Vị & Derangement

Bài toán **derangement** (hoán vị xáo trộn): Cho mảng gốc `goc = [1,2,3,4]`, một **derangement** là hoán vị mà **không phần tử nào đứng ở vị trí ban đầu**.

> *Ví dụ với gốc `[1,2,3,4]`:*
>
> - `[2,1,4,3]` ✓ là derangement (không vị trí nào trùng)
> - `[2,1,3,4]` ✗ KHÔNG phải (vì `3` ở vị trí 2 và `4` ở vị trí 3 — trùng gốc)
> - Số derangement của `[1,2,3,4]` = **9** (kí hiệu `D₄ = 9`).

a) (0.7đ) Quan sát code ở ảnh dưới. **Gõ lại toàn bộ và điền 7 chỗ trống** `___(n)___`. Không thêm/xóa dòng.

b) (0.3đ) Chạy `hoan_vi(arr=[1,2,3,4])` và đếm tổng số hoán vị in ra. Verify = `4! = 24`.

c) (0.6đ) Gọi `derangement([1,2,3,4], [1,2,3,4], 0, [])`. In tất cả derangement và đếm. Kiểm tra = **9**. In thêm số lần gọi đệ quy `dem_d`.

d) (0.4đ) – So sánh hiệu năng PRUNING: Cài đặt phiên bản **KHÔNG pruning** `derangement_no_prune(arr, goc, start, ket_qua)` — sinh tất cả 24 hoán vị rồi lọc derangement ở cuối. So sánh số lần gọi đệ quy với phiên bản có pruning. Giải thích bằng comment vì sao pruning hiệu quả hơn.

```
from IPython.display import Image, display
display(Image('cau1_code_lan3.png', width=1000))
```

CAU 1 - Dien vao 7 cho trong ___(n) (Quay Lui - Hoan Vi & Derangement)

```
# Quay Lui (Backtracking) - Sinh Hoan Vi & Bai Toan Derangement

dem_goi = 0

def hoan_vi(arr, start=0):
    """Sinh tat ca hoan vi cua arr (in-place swap)."""
    global dem_goi
    dem_goi += 1
    if start == ___(1)___:                          # TRONG 1
        print(arr.copy())
        return
    for i in range(___(2)___, len(arr)):            # TRONG 2
        arr[start], arr[i] = ___(3)___              # TRONG 3
        hoan_vi(arr, ___(4)___)                     # TRONG 4
        # QUAY LUI: undo swap (tra lai trang thai cu)
        arr[start], arr[i] = ___(5)___              # TRONG 5

# --- DERANGEMENT: hoan vi KHONG co phan tu nao o vi tri goc ---
# Vi du goc=[1,2,3,4]: [2,1,4,3] OK | [2,1,3,4] KO (3 va 4 con trung goc)

dem_d = 0

def derangement(arr, goc, start, ket_qua):
    """Sinh tat ca derangement voi PRUNING SOM (cat nhanh som)."""
    global dem_d
    dem_d += 1
    if start == len(arr):
        ket_qua.append(arr.copy())
        return
    for i in range(start, len(arr)):
        # PRUNING: phan tu arr[i] se trung vi tri goc -> bo qua nhanh nay
        if arr[i] == ___(6)___:                     # TRONG 6
            continue
        arr[start], arr[i] = arr[i], arr[start]
        derangement(arr, goc, ___(7)___, ket_qua)   # TRONG 7
        arr[start], arr[i] = arr[i], arr[start]
```

```
# Quay Lui (Backtracking) - Sinh Hoan Vi & Bai Toan Derangement

dem_goi = 0

def hoan_vi(arr, start=0):
    """Sinh tat ca hoan vi cua arr (in-place swap)."""
    global dem_goi
    dem_goi += 1

    if start == len(arr):                       # TRONG 1
        print(arr.copy())
        return

    for i in range(start, len(arr)):            # TRONG 2
        arr[start], arr[i] = arr[i], arr[start]       # TRONG 3
        hoan_vi(arr, start + 1)                 # TRONG 4

        # QUAY LUI: undo swap (tra lai trang thai cu)
```

```
        arr[start], arr[i] = arr[i], arr[start]       # TRONG 5


# --- DERANGEMENT: hoan vi KHONG co phan tu nao o vi tri goc ---
# Vi du goc=[1,2,3,4]: [2,1,4,3] OK | [2,1,3,4] KO (3 va 4 con trung goc)

dem_d = 0

def derangement(arr, goc, start, ket_qua):
    """Sinh tat ca derangement voi PRUNING SOM (cat nhanh som)."""
    global dem_d
    dem_d += 1

    if start == len(arr):
        ket_qua.append(arr.copy())
        return

    for i in range(start, len(arr)):
        # PRUNING: phan tu arr[i] se trung vi tri goc -> bo qua nhanh nay
        if arr[i] == goc[start]:               # TRONG 6
            continue

        arr[start], arr[i] = arr[i], arr[start]
        derangement(arr, goc, start + 1, ket_qua)     # TRONG 7
        arr[start], arr[i] = arr[i], arr[start]
```

```
print("Tat ca hoan vi cua [1,2,3,4]:")
print("-" * 30)
dem_goi = 0
def hoan_vi_with_count(arr, start=0):
    global dem_goi
    dem_goi += 1
    if start == len(arr):
        print(arr)
        return
    for i in range(start, len(arr)):
        arr[start], arr[i] = arr[i], arr[start]
        hoan_vi_with_count(arr, start + 1)
        arr[start], arr[i] = arr[i], arr[start]

hoan_vi_with_count([1, 2, 3, 4])
print("-" * 30)
print(f"So lan goi de quy hoan_vi: {dem_goi}")
print(f"4! = {4*3*2*1} (kiem tra so dong in ra phia tren)")
```

```
Tat ca hoan vi cua [1,2,3,4]:
------------------------------
[1, 2, 3, 4]
[1, 2, 4, 3]
[1, 3, 2, 4]
[1, 3, 4, 2]
[1, 4, 3, 2]
[1, 4, 2, 3]
[2, 1, 3, 4]
[2, 1, 4, 3]
[2, 3, 1, 4]
[2, 3, 4, 1]
[2, 4, 3, 1]
[2, 4, 1, 3]
[3, 2, 1, 4]
[3, 2, 4, 1]
[3, 1, 2, 4]
[3, 1, 4, 2]
[3, 4, 1, 2]
```

```
[3, 4, 2, 1]
[4, 2, 3, 1]
[4, 2, 1, 3]
[4, 3, 2, 1]
[4, 3, 1, 2]
[4, 1, 3, 2]
[4, 1, 2, 3]
------------------------------
So lan goi de quy hoan_vi: 65
4! = 24 (kiem tra so dong in ra phia tren)
```

```
# CAU 1c – Run derangement va dem
# ============================================================

dem_d = 0
ket_qua = []
goc = [1, 2, 3, 4]
derangement(goc.copy(), goc, 0, ket_qua)

print("Tat ca derangement cua [1,2,3,4]:")
for d in ket_qua:
    print(f"  {d}")
print("-" * 35)
print(f"So derangement (D_4): {len(ket_qua)}  (mong doi: 9)")
print(f"So lan goi de quy:    {dem_d}")
```

```
Tat ca derangement cua [1,2,3,4]:
  [2, 1, 4, 3]
  [2, 3, 4, 1]
  [2, 4, 1, 3]
  [3, 1, 4, 2]
  [3, 4, 1, 2]
  [3, 4, 2, 1]
  [4, 3, 2, 1]
  [4, 3, 1, 2]
  [4, 1, 2, 3]
-----------------------------------
So derangement (D_4): 9  (mong doi: 9)
So lan goi de quy:    31
```

```
dem_no = 0

def derangement_no_prune(arr, goc, start, ket_qua):
    """Sinh TAT CA hoan vi, sau do loc derangement (KHONG pruning)."""
    global dem_no
    dem_no += 1
    if start == len(arr):
        is_derangement = True
        for i in range(len(arr)):
            if arr[i] == goc[i]:
                is_derangement = False
                break
        if is_derangement:
            ket_qua.append(copy.deepcopy(arr))
        return
    for i in range(start, len(arr)):
        arr[start], arr[i] = arr[i], arr[start]
        derangement_no_prune(arr, goc, start + 1, ket_qua)
        arr[start], arr[i] = arr[i], arr[start]

dem_d = 0
ket_qua_prune = []
derangement([1,2,3,4], [1,2,3,4], 0, ket_qua_prune)

dem_no = 0
```

```
ket_qua_no = []
derangement_no_prune([1,2,3,4], [1,2,3,4], 0, ket_qua_no)

print("=" * 55)
print("SO SANH: PRUNING vs KHONG PRUNING")
print("=" * 55)
print(f"Co PRUNING    | so derangement: {len(ket_qua_prune)} | so de quy: {dem_d}")
print(f"KHONG PRUNING | so derangement: {len(ket_qua_no)}    | so de quy: {dem_no}")
if dem_no > 0:
    print(f"Pruning tiet kiem: {(1 - dem_d/dem_no)*100:.1f}% so lan goi")

# Giai thich:
# - KHONG pruning: Phai sinh ra toan bo 4! = 24 hoan vi (va tat ca cac nhanh trung gian) roi moi kiem t
# - CO pruning: Loai bo ngay cac nhanh ma tai vi tri hien tai da bi trung voi goc (arr[i] == goc[start]
```

```
=======================================================
SO SANH: PRUNING vs KHONG PRUNING
=======================================================
Co PRUNING    | so derangement: 9 | so de quy: 31
KHONG PRUNING | so derangement: 9    | so de quy: 65
Pruning tiet kiem: 52.3% so lan goi
```

CÂU 2 (4 ĐIỂM) – A*: Trace, Cài đặt, So sánh với UCS

a) (1.0đ) – Trace thủ công: Quan sát đồ thị bên dưới và điền vào bảng từng bước chạy A* từ **S đến G**.

b) (1.0đ) Cài đặt A* theo đúng đồ thị. Output phải **khớp** với bảng trace.

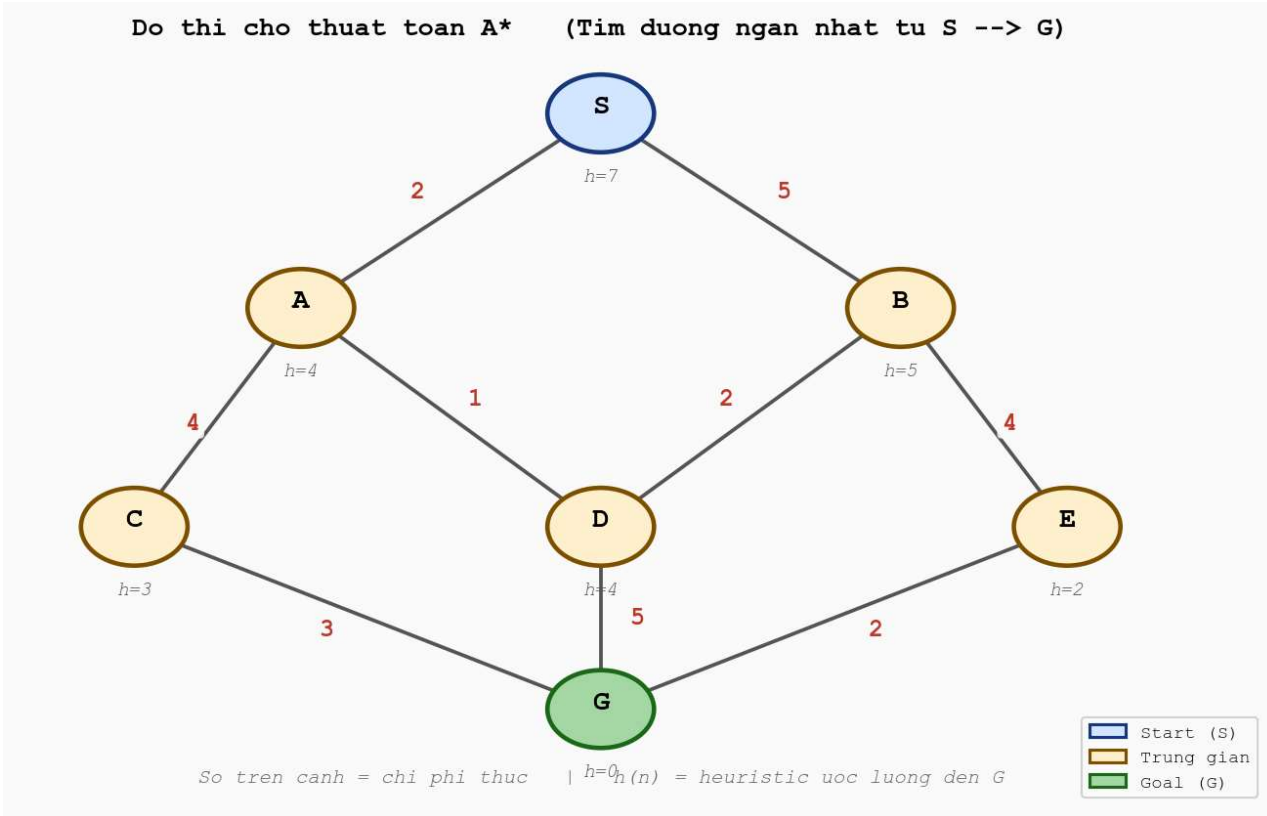
c) (0.5đ) Heuristic h(n) có **admissible** không? Kiểm tra `h(n) ≤ h*(n)` cho ít nhất **3 node** và kết luận.

*d) (1.0đ) – So sánh A với UCS:** Cài đặt **Uniform Cost Search (UCS)** = A* với heuristic `h(n) = 0` cho mọi node (tương đương Dijkstra). Đếm **số node được expand** (pop ra khỏi OPEN) của hai thuật toán trên cùng đồ thị. Lập bảng so sánh và nhận xét.

e) (0.5đ) – What-if cost: Tăng chi phí cạnh `A-D` từ **1** lên **6**. Chạy lại A*. Đường đi tối ưu có thay đổi không? Tổng chi phí mới là bao nhiêu?

```
Bắt đầu lập trình hoặc tạo mã bằng trí tuệ nhân tạo (AI).
```

```
from IPython.display import Image, display
display(Image('cau2_graph_lan3.png', width=850))
```



*Câu 2a – Bảng trace A (điền tay):**

Bước	Node xét	g(n)	h(n)	f(n)	OPEN (node:f) sau bước này	CLOSE
1	S	0	7	7	A:6, B:10	{S}
2	A	2	4	6	D(7),C(9),B(10)	{S,A}
3	D	3	4	7	G(8),B(10)	{S,A,D}
4	G	8	0	8	B(10)	{S,A,D,G}

Đường đi tìm được: S → A → D → G
Tổng chi phí thực: *8

Câu 2a – What-if (0.25đ bonus):

> Giả sử đổi `h(D) = 8` (thay vì 4). Lúc này h(D) = 8 > h*(D) = 5 → **không admissible** nữa.
> A* có còn tìm được đường đi **tối ưu** không? Vì sao?

(Viết câu trả lời ở đây)

```
import heapq

# TODO: Tu go lai graph tu do thi (7 nut: S, A, B, C, D, E, G)
graph = {
    'S': {'A': 2, 'B': 5},
    'A': {'S': 2, 'C': 4, 'D': 1},
    'B': {'S': 5, 'E': 4,'D':2},
    'C': {'A': 4, 'G': 3},
    'D': {'A': 1, 'G': 5,'B':2},
    'E': {'B': 4, 'G': 2},
    'G': {'C': 3, 'D': 5, 'E': 2} # G is connected back for completeness, though pathfinding is S->G
}
```

```
# TODO: Tu go lai heuristic tu do thi
heuristic = {
    'S': 7,
    'A': 4,
    'B': 5,
    'C': 3,
    'D': 4,
    'E': 2,
    'G': 0
}

def astar(graph, heuristic, start, goal):
    """
    Thuat toan A*. Tra ve (duong_di, tong_chi_phi, so_node_expand).
    In tung buoc:
      Buoc X | Xet: <node> | g=? h=? f=? | OPEN: [<node>:f, ...]
    """
    open_list = [] # Priority queue: (f_score, node)
    heapq.heappush(open_list, (heuristic[start], start)) # Initial f_score is 0 + h(start)

    came_from = {} # Dictionary to reconstruct path
    g_score = {node: float('inf') for node in graph} # Cost from start to current node
    g_score[start] = 0

    f_score = {node: float('inf') for node in graph} # Estimated total cost from start to goal through
    f_score[start] = heuristic[start]

    closed_list = set() # Set of nodes already evaluated (expanded)
    so_node_expand = 0
    step_count = 0

    # Print header for trace table
    print("| Bước | Node xét | g(n) | h(n) | f(n) | OPEN (node:f) sau bước này       | CLOSE       |")
    print("|------|----------|------|------|------|----------------------------------|-------------|")

    while open_list:
        current_f, current_node = heapq.heappop(open_list)

        # If we already processed this node or found a better path, skip this outdated entry
        if current_node in closed_list:
            continue

        step_count += 1
        so_node_expand += 1
        closed_list.add(current_node)

        # Print current node details
        print(f"| {step_count:<4} | {current_node:<8} | {g_score[current_node]:<4} | {heuristic[curren

        if current_node == goal:
            path = []
            temp_node = current_node
            while temp_node in came_from:
                path.insert(0, temp_node)
                temp_node = came_from[temp_node]
            path.insert(0, start)

            # Prepare OPEN and CLOSE for final step print
            open_display_items = []
            for node_in_graph in graph:
                if node_in_graph not in closed_list and g_score[node_in_graph] != float('inf'):
                    open_display_items.append(f"{node_in_graph}:{f_score[node_in_graph]:g}")
```

```
            open_display = ", ".join(sorted(open_display_items, key=lambda x: (float(x.split(':')[1]),
            close_display = "{" + ", ".join(sorted(list(closed_list))) + "}"
            print(f"{open_display:<32} | {close_display:<11} |")

            return path, g_score[goal], so_node_expand

        # Explore neighbors
        for neighbor, weight in graph[current_node].items():
            # Skip if neighbor is already in closed_list to avoid reprocessing optimal path
            if neighbor in closed_list:
                continue

            tentative_g_score = g_score[current_node] + weight

            # If a shorter path to neighbor is found or it's a new node
            if tentative_g_score < g_score[neighbor]:
                came_from[neighbor] = current_node
                g_score[neighbor] = tentative_g_score
                f_score[neighbor] = g_score[neighbor] + heuristic[neighbor]
                heapq.heappush(open_list, (f_score[neighbor], neighbor))

        # After processing all neighbors, prepare the OPEN list for printing
        # This list should include all nodes that have been discovered but not yet expanded (not in cl
        # and for which a path has been found (g_score is not infinity).
        open_display_items = []
        # Collect current items in open_list without duplicates (based on node) and with correct f_sco
        current_open_nodes_info = {} # node -> f_score
        for f_val, node in open_list:
            # Ensure we take the lowest f_score if a node has multiple entries due to updates
            if node not in current_open_nodes_info or f_val < current_open_nodes_info[node]:
                current_open_nodes_info[node] = f_val

        for node_in_open, f_val in current_open_nodes_info.items():
            if node_in_open not in closed_list: # Ensure it's not already in closed_list
                open_display_items.append(f"{node_in_open}:{f_val:g}")

        # Sort by f_score, then alphabetically by node name for consistent display
        open_display = ", ".join(sorted(open_display_items, key=lambda x: (float(x.split(':')[1]), x.s
        close_display = "{" + ", ".join(sorted(list(closed_list))) + "}"
        print(f"{open_display:<32} | {close_display:<11} |")

    return None, float('inf'), so_node_expand

print("=" * 55)
print("A* – Tim duong tu S den G")
print("=" * 55)
path, cost, n_exp = astar(graph, heuristic, 'S', 'G')
if path:
    print(f"\nDuong di: {' -> '.join(path)}")
    print(f"Tong chi phi: {cost}")
    print(f"So node expand: {n_exp}")
```

```
=======================================================
A* – Tim duong tu S den G
=======================================================
| Bước | Node xét | g(n) | h(n) | f(n) | OPEN (node:f) sau bước này       | CLOSE       |
|------|----------|------|------|------|----------------------------------|-------------|
| 1    | S        | 0    | 7    | 7    | A:6, B:10                        | {S}         |
| 2    | A        | 2    | 4    | 6    | D:7, C:9, B:10                   | {A, S}      |
| 3    | D        | 3    | 4    | 7    | G:8, C:9, B:10                   | {A, D, S}   |
| 4    | G        | 8    | 0    | 8    | C:9, B:10                        | {A, D, G, S} |

Duong di: S -> A -> D -> G
```

```
Tong chi phi: 8
So node expand: 4
```

Câu 2c – Kiểm tra admissible:

(Tính h(n) = chi phí thực tế ngắn nhất từ n đến G)*

- h*(S) = 8 (S -> A -> D -> G: 2 + 1 + 5) ⟶ h(S)=7 ⟶ h(S) ≤ h*(S)? : Có (7 <= 8)
- h*(A) = 6 (A -> D -> G: 1 + 5) ⟶ h(A)=4 ⟶ h(A) ≤ h*(A)? : Có (4 <= 6)
- h*(D) = 5 (D -> G: 5) ⟶ h(D)=4 ⟶ h(D) ≤ h*(D)? : Có (4 <= 5)
- h*(B) = 6 (B -> E -> G: 4 + 2) ⟶ h(B)=5 ⟶ h(B) ≤ h*(B)? : Có (5 <= 6)

Kết luận: Heuristic `h(n)` là **admissible** vì `h(n) <= h*(n)` cho tất cả các node được kiểm tra (và trong thực tế, cho tất cả các node của đồ thị này).

```
# CAU 2d – Cai dat UCS (Uniform Cost Search) va so sanh voi A*
# UCS = A* voi h(n) = 0 cho moi node (tuong duong Dijkstra)
# ============================================================

import heapq

def ucs(graph, start, goal):
    """
    Uniform Cost Search.
    Chi dung g(n) - khong dung heuristic.
    Tra ve (duong_di, tong_chi_phi, so_node_expand).
    """
    open_list = [] # Priority queue: (g_score, node)
    heapq.heappush(open_list, (0, start)) # Initial g_score is 0

    came_from = {} # Dictionary to reconstruct path
    g_score = {node: float('inf') for node in graph} # Cost from start to current node
    g_score[start] = 0

    closed_list = set() # Set of nodes already evaluated (expanded)
    so_node_expand = 0

    while open_list:
        current_g, current_node = heapq.heappop(open_list)

        if current_node in closed_list:
            continue

        so_node_expand += 1
        closed_list.add(current_node)

        if current_node == goal:
            path = []
            temp_node = current_node
            while temp_node in came_from:
                path.insert(0, temp_node)
                temp_node = came_from[temp_node]
            path.insert(0, start)
            return path, g_score[goal], so_node_expand

        for neighbor, weight in graph[current_node].items():
            if neighbor in closed_list:
                continue

            tentative_g_score = g_score[current_node] + weight
```

```
            if tentative_g_score < g_score[neighbor]:
                came_from[neighbor] = current_node
                g_score[neighbor] = tentative_g_score
                heapq.heappush(open_list, (g_score[neighbor], neighbor))

    return None, float('inf'), so_node_expand


# So sanh A* va UCS
path_a, cost_a, exp_a = astar(graph, heuristic, 'S', 'G')
path_u, cost_u, exp_u = ucs(graph, 'S', 'G')

print("=" * 55)
print("SO SANH A* vs UCS")
print("=" * 55)
print(f"{'Thuat toan':<12} {'Duong di':<22} {'Cost':<6} {'#Expand':<10}")
print("-" * 55)
print(f"{'A*':<12} {' -> '.join(path_a):<22} {cost_a:<6} {exp_a:<10}")
print(f"{'UCS':<12} {' -> '.join(path_u):<22} {cost_u:<6} {exp_u:<10}")

# TODO (comment): Nhan xet
# - Hai thuat toan co cung tong chi phi khong?
#   Có, cả hai thuật toán (A* với heuristic admissible và UCS) đều tìm thấy đường đi tối ưu, nên tổng
# - Thuat toan nao expand it node hon? Vi sao?
#   A* thường expand ít node hơn UCS. Vì A* sử dụng hàm heuristic để ước lượng chi phí còn lại đến đíc
# - Khi nao A* va UCS cho ket qua giong nhau?
#   A* và UCS sẽ cho kết quả giống nhau (cả về đường đi và chi phí) khi hàm heuristic h(n) luôn bằng 6
```

```
| Bước | Node xét | g(n) | h(n) | f(n) | OPEN (node:f) sau bước này       | CLOSE       |
|------|----------|------|------|------|----------------------------------|-------------|
| 1    | S        | 0    | 7    | 7    | A:6, B:10                        | {S}         |
| 2    | A        | 2    | 4    | 6    | D:7, C:9, B:10                   | {A, S}      |
| 3    | D        | 3    | 4    | 7    | G:8, C:9, B:10                   | {A, D, S}   |
| 4    | G        | 8    | 0    | 8    | C:9, B:10                        | {A, D, G, S} |
=======================================================
SO SANH A* vs UCS
=======================================================
Thuat toan   Duong di               Cost   #Expand
-------------------------------------------------------
A*           S -> A -> D -> G       8      4
UCS          S -> A -> D -> G       8      6
```

```
# CAU 2e – What-if: tang cost(A,D) tu 1 len 6
# ============================================================
import copy
graph_v2 = copy.deepcopy(graph)

# TODO: Cap nhat trong so canh A-D thanh 6 (ca hai chieu vi do thi vo huong)
graph_v2['A']['D'] = 6
graph_v2['D']['A'] = 6

print("=" * 55)
print("A* sau khi tang cost(A,D) tu 1 -> 6")
print("=" * 55)
path_v2, cost_v2, _ = astar(graph_v2, heuristic, 'S', 'G')
print(f"Duong di moi: {' -> '.join(path_v2)}")
print(f"Tong chi phi: {cost_v2}")

# TODO (comment): Duong di co thay doi khong? Vi sao?
# Duong di co thay doi. Ban dau, duong S -> A -> D -> G co chi phi la 2 + 1 + 5 = 8. Sau khi tang cost(
# chi phi cua duong nay tro thanh 2 + 6 + 5 = 13.
# Luc nay, A* se tim duoc duong di toi uu khac voi chi phi thap hon, cu the la S -> B -> E -> G voi chi
# Su thay doi chi phi mot canh quan trong co the lam thay doi hoan toan duong di toi uu ma thuat toan t
```

```
=======================================================
A* sau khi tang cost(A,D) tu 1 -> 6
=======================================================
| Bước | Node xét | g(n) | h(n) | f(n) | OPEN (node:f) sau bước này       | CLOSE       |
|------|----------|------|------|------|----------------------------------|-------------|
| 1    | S        | 0    | 7    | 7    | A:6, B:10                        | {S}         |
| 2    | A        | 2    | 4    | 6    | C:9, B:10, D:12                  | {A, S}      |
| 3    | C        | 6    | 3    | 9    | G:9, B:10, D:12                  | {A, C, S}   |
| 4    | G        | 9    | 0    | 9    | B:10, D:12                       | {A, C, G, S} |
Duong di moi: S -> A -> C -> G
Tong chi phi: 9
```

CÂU 3 (4 ĐIỂM) – Minimax cho TicTacToe + Vòng Lặp Game

Phần A – Cài đặt Minimax (2.0đ)

a) (1.5đ) Quan sát code ở ảnh dưới. **Gõ lại toàn bộ và điền 7 chỗ trống** `___(n)___`.

b) (0.5đ) – Tính tay state #1: Cho trạng thái (lượt **X** đi):

```
X | X | _
O | O | _
_ | _ | _
```

Không chạy code, **tính tay** xem Minimax chọn ô nào cho lượt X. *(Gợi ý: X có nước **thắng ngay**, nhưng nếu KHÔNG chọn nó thì O sẽ thắng ở lượt sau!)*

Phần B – Áp dụng (2.0đ)

c) (0.5đ) – Tính tay state #2: Cho trạng thái (lượt **O** đi):

```
X | X | _
_ | O | _
_ | _ | _
```

Không chạy code, tính tay nước đi của O. (Gợi ý: O phải **chặn** X khỏi thắng. X đang đe dọa thắng ở ô nào?)

d) (1.0đ) – Vòng lặp game: Cài đặt hàm `play_game()` cho phép **người chơi (X) đấu với AI (O)**:

- In bảng đầu game.
- Người nhập tọa độ theo cú pháp `"hàng cột"` (ví dụ `"1 2"`).
- AI dùng `minimax()` chọn nước đi tối ưu.
- In bảng sau mỗi lượt.
- Kiểm tra input không hợp lệ (ngoài bàn / ô đã đánh) → yêu cầu nhập lại.
- Khi `terminal(board)` → in kết quả: `X thắng / O thắng / Hòa`.

e) (0.5đ) Tại sao Minimax chậm với bàn lớn hơn 3×3? Đề xuất **một cải tiến** cụ thể (mô tả ngắn, không cần code).

```
from IPython.display import Image, display
display(Image('cau3_code_lan3.png', width=1000))
```

CAU 3 - Dien vao 7 cho trong ___(n) (khong them / xoa dong)

```
import copy, math
X, O, EMPTY = 'X', 'O', None

def initial_state():
    return [[EMPTY]*3 for _ in range(3)]

def player(board):
    x_count = sum(row.count(X) for row in board)
    o_count = sum(row.count(O) for row in board)
    return X if x_count == o_count else ___(1)___    # TRONG 1
```

```
# CAU 3a – Nhin anh phia tren, go lai toan bo code va dien 7 cho trong
# ============================================================
import copy, math

X, O, EMPTY = "X", "O", None

# TODO: Go lai va dien 7 cho trong
def initial_state():
  return [[EMPTY]*3 for _ in range(3)]

def player(board):
  x_count = sum(row.count(X) for row in board)
  o_count = sum(row.count(O) for row in board)
  return X if x_count == o_count else O

def actions(board):
  return [(i, j) for i in range(3) for j in range(3) if board[i][j] == EMPTY]

def result(board, action):
    i, j = action
    if board[i][j] != EMPTY:
        raise ValueError("Invalid action: cell not empty")
    new_board = copy.deepcopy(board)
    new_board[i][j] = player(board)
    return new_board

def winner(board):
    # Check rows
    for row in board:
        if row[0] == row[1] == row[2] and row[0] != EMPTY:
            return row[0]
    # Check columns
    for col in range(3):
        if board[0][col] == board[1][col] == board[2][col] and board[0][col] != EMPTY:
            return board[0][col]
    # Check diagonals
    if board[0][0] == board[1][1] == board[2][2] and board[0][0] != EMPTY:
        return board[0][0]
    if board[0][2] == board[1][1] == board[2][0] and board[0][2] != EMPTY:
        return board[0][2]
    return None

def terminal(board):
    if winner(board) is not None:
        return True
    if EMPTY not in [cell for row in board for cell in row]:
        return True
```

```
        return False
    def utility(board):
       return 1 if winner(board) == X else -1 if winner(board) == O else 0
    def max_value(state):
      if terminal(state):return utility(state)
      v = -math.inf
      for action in actions(state):
        v = max(v, min_value(result(state, action)))
      return v

    def min_value(state):
      if terminal(state):return utility(state)
      v = +math.inf
      for action in actions(state):
        v = max(v, max_value(result(state, action)))
      return v
```

Câu 3b – Tính tay state #1 (lượt X):

Bảng:

```
X | X | _
O | O | _
_ | _ | _
```

Ô trống: `(0,2), (1,2), (2,0), (2,1), (2,2)`

Phân tích từng nước đi của X:

- **X đi** `(0,2)` → hàng 0 = `X,X,X` → **X thắng ngay** → utility = `+1`
- **X đi** `(1,2)` → chặn O (vì O đe dọa hàng 1). Game tiếp tục. Nếu O chơi tối ưu, O sẽ đi `(0,2)` để chặn X, dẫn đến **HÒA** → utility = `0`
- **X đi** `(2,0)`, `(2,1)`, **hoặc** `(2,2)` → KHÔNG chặn O → O sẽ đi `(1,2)` → hàng 1 = `O,O,O` → **O thắng** → utility = `-1`

X là MAX → chọn nước có `min_value` LỚN nhất. **Kết luận:** Minimax chọn ô **(0,2)**

Câu 3c – Tính tay state #2 (lượt O):

Bảng:

```
X | X | _
_ | O | _
_ | _ | _
```

X đang đe dọa thắng tại ô **(0,2)** (X có 2 quân liên tiếp ở hàng 0).

Nếu O **không chặn**, lượt sau X sẽ thắng → utility = +1 → tệ cho O. Vậy O (MIN) phải chọn ô để **chặn** X.

Kết luận: Minimax (O) chọn ô **(0,2)**

```
    # CAU 3d – Cai dat vong lap game (nguoi vs AI)
    # ============================================================

    def print_board(board):
        """In bang co"""
        print("  0 1 2")
        for i, row in enumerate(board):
```

```
        print(f"{i} " + " ".join(c if c else '.' for c in row))
    print()

def play_game():
    """
    Vong lap game: Nguoi (X) vs AI (O).
    - Nguoi nhap toa do dang 'hang cot' (vd: '1 2')
    - AI dung minimax() chon nuoc di toi uu
    - In bang sau moi luot, kiem tra ket thuc
    """
    board = initial_state()
    print("=" * 40)
    print("TICTACTOE – Nguoi (X) vs AI (O)")
    print("Nhap toa do dang: hang cot (vd: 1 2)")
    print("=" * 40)
    print_board(board)

    while not terminal(board):
        current = player(board)
        if current == X:
            while True:
                try:
                    user_input = input(f"Luot {current} di (hang cot, vd: 1 2): ").strip().split()
                    row, col = int(user_input[0]), int(user_input[1])
                    action = (row, col)

                    if not (0 <= row < 3 and 0 <= col < 3):
                        print("Toa do khong hop le. Vui long nhap lai trong pham vi 0-2.")
                        continue
                    if board[row][col] != EMPTY:
                        print("O nay da duoc danh. Vui long chon o khac.")
                        continue

                    board = result(board, action)
                    break
                except (ValueError, IndexError):
                    print("Dau vao khong hop le. Vui long nhap theo dinh dang 'hang cot' (vd: 1 2).")
        else:
            print(f"Luot {current} (AI) dang suy nghi...")
            ai_action = pruning_minimax(board)
            if ai_action:
                print(f"AI di o {ai_action}")
                board = result(board, ai_action)
            else:
                print("AI khong tim thay nuoc di hop le. Game ket thuc.")
                break

        print_board(board)

    game_winner = winner(board)
    if game_winner == X:
        print("X thang!")
    elif game_winner == O:
        print("O thang!")
    else:
        print("Hoa!")

# Bo comment dong duoi de chay (tat khi nop bai)
play_game()
```

```
========================================
TICTACTOE – Nguoi (X) vs AI (O)
Nhap toa do dang: hang cot (vd: 1 2)
========================================
```

```
  0 1 2
0 . . .
1 . . .
2 . . .

Luot X di (hang cot, vd: 1 2): 11
Dau vao khong hop le. Vui long nhap theo dinh dang 'hang cot' (vd: 1 2).
Luot X di (hang cot, vd: 1 2): 1 1
  0 1 2
0 . . .
1 . X .
2 . . .

Luot O (AI) dang suy nghi...
AI di o (0, 0)
  0 1 2
0 O . .
1 . X .
2 . . .

Luot X di (hang cot, vd: 1 2): 0 2
  0 1 2
0 O . X
1 . X .
2 . . .

Luot O (AI) dang suy nghi...
AI di o (2, 0)
  0 1 2
0 O . X
1 . X .
2 O . .

Luot X di (hang cot, vd: 1 2): 0 1
  0 1 2
0 O X X
1 . X .
2 O . .

Luot O (AI) dang suy nghi...
AI di o (1, 0)
  0 1 2
0 O X X
1 O X .
2 O . .

O thang!
```

Câu 3e – Tại sao Minimax chậm và đề xuất cải tiến:

- **Tại sao Minimax chậm:**
  - **Branching Factor (Hệ số nhánh):** Với bàn cờ lớn hơn (ví dụ 4x4 hoặc 5x5), số lượng nước đi hợp lệ tại mỗi bước tăng lên đáng kể.
  - **Độ sâu cây:** Minimax duyệt toàn bộ cây trò chơi. Số lượng trạng thái cần kiểm tra tăng theo cấp số nhân (exponentially) theo công thức b^d (với b là hệ số nhánh và d là độ sâu). Ví dụ bàn 3x3 có tối đa 9! trạng thái, nhưng bàn 4x4 có tới 16!, một con số khổng lồ cho máy tính xử lý trong thời gian thực.
- **Đề xuất cải tiến:**
  - **Cắt tỉa Alpha-Beta (Alpha-Beta Pruning):** Đây là kỹ thuật giúp loại bỏ các nhánh của cây trò chơi mà chắc chắn không ảnh hưởng đến kết quả cuối cùng, giúp giảm đáng kể số lượng node cần duyệt mà vẫn tìm được nước đi tối ưu.
  - **Giới hạn độ sâu (Depth-limited search):** Thay vì duyệt đến tận trạng thái kết thúc (terminal state), thuật toán chỉ duyệt đến một độ sâu nhất định (ví dụ 5-6 lớp) và sử dụng một **Hàm đánh giá**

(Heuristic Evaluation Function) để ước lượng giá trị của trạng thái tại đó.

```
# ============================================================
# CELL NOP BAI – Chay sau khi hoan thanh toan bo de
# Sinh vien KHONG sua cell nay
# ============================================================
import os, subprocess

try:
    from google.colab import drive
    IS_COLAB = True
except ImportError:
    IS_COLAB = False

NOTEBOOK_NAME = "De_Thi_Lan3.ipynb"

print("Buoc 1: Dang chuyen doi sang PDF...")

result = subprocess.run(
    ["jupyter", "nbconvert", "--to", "webpdf",
     "--allow-chromium-download", "--no-input",
     NOTEBOOK_NAME],
    capture_output=True, text=True
)

pdf_file = NOTEBOOK_NAME.replace(".ipynb", ".pdf")

if result.returncode != 0 or not os.path.exists(pdf_file):
    print("  webpdf that bai, chuyen sang HTML...")
    subprocess.run(
        ["jupyter", "nbconvert", "--to", "html", "--no-input", NOTEBOOK_NAME],
        capture_output=True
    )
    html_file = NOTEBOOK_NAME.replace(".ipynb", ".html")
    if os.path.exists(html_file):
        if IS_COLAB:
            from google.colab import files
            files.download(html_file)
        print(f"\n[HTML] File '{html_file}' da tai xuong.")
        print("=> Mo file HTML trong trinh duyet -> Ctrl+P -> Save as PDF")
    else:
        print("Loi: Khong the chuyen doi. Hay dung File > Print trong Colab.")
else:
    print(f"  Thanh cong! File: {pdf_file}")
    if IS_COLAB:
        from google.colab import files
        files.download(pdf_file)
        print(f"\n[PDF] '{pdf_file}' da tai xuong may tinh.")
    else:
        print(f"\n[PDF] Da luu tai: {os.path.abspath(pdf_file)}")

print("\nDat ten lai file theo dinh dang: MSSV_HoTen.pdf")
print("Nop qua link giao vien da cung cap.")
```

```
Buoc 1: Dang chuyen doi sang PDF...
  webpdf that bai, chuyen sang HTML...
Loi: Khong the chuyen doi. Hay dung File > Print trong Colab.

Dat ten lai file theo dinh dang: MSSV_HoTen.pdf
Nop qua link giao vien da cung cap.
```